ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 21

Кэширование с использованием Redis

Рассмотрим процесс работы с Redis и применением Redis для кэширования. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Заголовок

Redis — это быстрое in-memory хранилище данных типа key-value, которое часто используется для кэширования, хранения сессий, очередей сообщений и временных данных.

Кэширование позволяет сохранять результаты часто выполняемых запросов на некоторое время, чтобы при повторном обращении не выполнять одну и ту же логику повторно. Это особенно полезно для маршрутов чтения данных, например получения списка пользователей или товаров.

Преимущества кэширования:

- уменьшение нагрузки на сервер и базу данных;
- ускорение ответа клиенту;
- повышение масштабируемости приложения;
- снижение количества одинаковых вычислений и запросов.

При использовании Redis схема работы выглядит следующим образом:

1. Клиент отправляет запрос на сервер.
2. Сервер проверяет, есть ли нужные данные в Redis.

3. Если данные найдены, они возвращаются из кэша.
4. Если данных нет, сервер получает их обычным способом, отправляет клиенту и сохраняет в Redis на заданное время.

Таким образом, кэш выступает промежуточным слоем между клиентом и основным источником данных.

Реализация кэширования на сервере

Доработаем приложение из практического занятия №11. Для этого:

- подключим Redis;
- создадим middleware для чтения данных из кэша;
- создадим функцию сохранения данных в кэш;
- добавим очистку кэша при изменении пользователей и товаров.

Установим необходимые зависимости:

```
npm install redis
```

Приведём пример приложения с кешированием на основе знакомой нам структуры RBAC:

```
const express = require("express");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const { createClient } = require("redis");

const app = express();
app.use(express.json());

const PORT = 3000;

// Секреты подписи
const ACCESS_SECRET = "access_secret";
const REFRESH_SECRET = "refresh_secret";

// Время жизни токенов
const ACCESS_EXPIRES_IN = "15m";
const REFRESH_EXPIRES_IN = "7d";

// Время хранения кэша
const USERS_CACHE_TTL = 60;       // 1 минута
const PRODUCTS_CACHE_TTL = 600;   // 10 минут
```

```
// { id, username, passwordHash, role, blocked }
const users = [];

// { id, name, price, description }
const products = [];

// Хранилище refresh-токенов
const refreshTokens = new Set();

// Redis client
const redisClient = createClient({
  url: "redis://127.0.0.1:6379"
});

redisClient.on("error", (err) => {
  console.error("Redis error:", err);
});

async function initRedis() {
  await redisClient.connect();
  console.log("Redis connected");
}

function generateAccessToken(user) {
  return jwt.sign(
    {
      sub: user.id,
      username: user.username,
      role: user.role
    },
    ACCESS_SECRET,
    {
      expiresIn: ACCESS_EXPIRES_IN,
    }
  );
}

function generateRefreshToken(user) {
  return jwt.sign(
    {
      sub: user.id,
      username: user.username,
      role: user.role
    },
    REFRESH_SECRET,
    {
```

```
      expiresIn: REFRESH_EXPIRES_IN,
    }
  );
}

// Auth middleware
function authMiddleware(req, res, next) {
  const header = req.headers.authorization || "";
  const [scheme, token] = header.split(" ");

  if (scheme !== "Bearer" || !token) {
    return res.status(401).json({
      error: "Missing or invalid Authorization header",
    });
  }

  try {
    const payload = jwt.verify(token, ACCESS_SECRET);

    const user = users.find((u) => u.id === payload.sub);
    if (!user || user.blocked) {
      return res.status(401).json({
        error: "User not found or blocked",
      });
    }

    req.user = payload;
    next();
  } catch (err) {
    return res.status(401).json({
      error: "Invalid or expired token",
    });
  }
}

// Role middleware
function roleMiddleware(allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        error: "Forbidden",
      });
    }
    next();
  };
}
```

```
// Middleware чтения из кэша
function cacheMiddleware(keyBuilder, ttl) {
  return async (req, res, next) => {
    try {
      const key = keyBuilder(req);
      const cachedData = await redisClient.get(key);

      if (cachedData) {
        return res.json({
          source: "cache",
          data: JSON.parse(cachedData)
        });
      }

      req.cacheKey = key;
      req.cacheTTL = ttl;
      next();
    } catch (err) {
      console.error("Cache read error:", err);
      next();
    }
  };
}

// Сохранение ответа в кэш
async function saveToCache(key, data, ttl) {
  try {
    await redisClient.set(key, JSON.stringify(data), {
      EX: ttl
    });
  } catch (err) {
    console.error("Cache save error:", err);
  }
}

// Удаление кэша пользователей
async function invalidateUsersCache(userId = null) {
  try {
    await redisClient.del("users:all");
    if (userId) {
      await redisClient.del(`users:${userId}`);
    }
  } catch (err) {
    console.error("Users cache invalidate error:", err);
  }
```

```
}

// ---------------- AUTH ----------------

app.post("/api/auth/register", async (req, res) => {
  const { username, password, role } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
    });
  }

  const exists = users.some((u) => u.username === username);
  if (exists) {
    return res.status(409).json({
      error: "username already exists",
    });
  }

  const passwordHash = await bcrypt.hash(password, 10);

  const user = {
    id: String(users.length + 1),
    username,
    passwordHash,
    role: role || "user",
    blocked: false
  };

  users.push(user);

  res.status(201).json({
    id: user.id,
    username: user.username,
    role: user.role,
    blocked: user.blocked
  });
});

app.post("/api/auth/login", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      error: "username and password are required",
```

```
    });
  }

  const user = users.find((u) => u.username === username);
  if (!user || user.blocked) {
    return res.status(401).json({
      error: "Invalid credentials or user is blocked",
    });
  }

  const isValid = await bcrypt.compare(password, user.passwordHash);
  if (!isValid) {
    return res.status(401).json({
      error: "Invalid credentials",
    });
  }

  const accessToken = generateAccessToken(user);
  const refreshToken = generateRefreshToken(user);

  refreshTokens.add(refreshToken);

  res.json({
    accessToken,
    refreshToken,
  });
});

app.post("/api/auth/refresh", (req, res) => {
  const { refreshToken } = req.body;

  if (!refreshToken) {
    return res.status(400).json({
      error: "refreshToken is required",
    });
  }

  if (!refreshTokens.has(refreshToken)) {
    return res.status(401).json({
      error: "Invalid refresh token",
    });
  }

  try {
    const payload = jwt.verify(refreshToken, REFRESH_SECRET);
```

```
    const user = users.find((u) => u.id === payload.sub);
    if (!user || user.blocked) {
      return res.status(401).json({
        error: "User not found or blocked",
      });
    }

    refreshTokens.delete(refreshToken);

    const newAccessToken = generateAccessToken(user);
    const newRefreshToken = generateRefreshToken(user);

    refreshTokens.add(newRefreshToken);

    res.json({
      accessToken: newAccessToken,
      refreshToken: newRefreshToken,
    });
  } catch (err) {
    return res.status(401).json({
      error: "Invalid or expired refresh token",
    });
  }
});

app.get("/api/auth/me", authMiddleware, roleMiddleware(["user", "seller",
"admin"]), (req, res) => {
  const user = users.find((u) => u.id === req.user.sub);

  res.json({
    id: user.id,
    username: user.username,
    role: user.role,
    blocked: user.blocked
  });
});

// ---------------- USERS ----------------

// Получить список пользователей (кэш 1 минута)
app.get(
  "/api/users",
  authMiddleware,
  roleMiddleware(["admin"]),
  cacheMiddleware(() => "users:all", USERS_CACHE_TTL),
  async (req, res) => {
```

```
    const data = users.map((u) => ({
      id: u.id,
      username: u.username,
      role: u.role,
      blocked: u.blocked
    }));

    await saveToCache(req.cacheKey, data, req.cacheTTL);

    res.json({
      source: "server",
      data
    });
  }
);

// Получить пользователя по id (кэш 1 минута)
app.get(
  "/api/users/:id",
  authMiddleware,
  roleMiddleware(["admin"]),
  cacheMiddleware((req) => `users:${req.params.id}`, USERS_CACHE_TTL),
  async (req, res) => {
    const user = users.find((u) => u.id === req.params.id);

    if (!user) {
      return res.status(404).json({
        error: "User not found"
      });
    }

    const data = {
      id: user.id,
      username: user.username,
      role: user.role,
      blocked: user.blocked
    };

    await saveToCache(req.cacheKey, data, req.cacheTTL);

    res.json({
      source: "server",
      data
    });
  }
);
```

```
// Обновить пользователя
app.put("/api/users/:id", authMiddleware, roleMiddleware(["admin"]), async
(req, res) => {
  const { username, role, blocked } = req.body;
  const user = users.find((u) => u.id === req.params.id);

  if (!user) {
    return res.status(404).json({
      error: "User not found"
    });
  }

  if (username !== undefined) user.username = username;
  if (role !== undefined) user.role = role;
  if (blocked !== undefined) user.blocked = blocked;

  await invalidateUsersCache(user.id);

  res.json({
    id: user.id,
    username: user.username,
    role: user.role,
    blocked: user.blocked
  });
});

// Заблокировать пользователя
app.delete("/api/users/:id", authMiddleware, roleMiddleware(["admin"]), async
(req, res) => {
  const user = users.find((u) => u.id === req.params.id);

  if (!user) {
    return res.status(404).json({
      error: "User not found"
    });
  }

  user.blocked = true;

  await invalidateUsersCache(user.id);

  res.json({
    message: "User blocked",
    id: user.id
  });
```

```
});

initRedis().then(() => {
  app.listen(PORT, () => {
    console.log(`Сервер запущен на http://localhost:${PORT}`);
  });
});
```

Пояснение к реализации

В данной версии приложения Redis используется для кэширования следующих маршрутов:

- `GET /api/users` — список пользователей хранится в кэше **1 минуту**;
- `GET /api/users/:id` — пользователь по id хранится в кэше **1 минуту**;
- `GET /api/products` — список товаров хранится в кэше **10 минут**;
- `GET /api/products/:id` — товар по id хранится в кэше **10 минут**.

Для этого был создан `cacheMiddleware`, который:

1. формирует ключ кэша;
2. пытается получить значение из Redis;
3. если данные найдены — сразу возвращает ответ;
4. если данные не найдены — передаёт управление следующему обработчику.

После получения данных обычным способом они сохраняются в Redis с помощью функции `saveToCache`.

Кроме того, при изменении пользователей и товаров кэш должен очищаться, иначе клиент может получить устаревшие данные. Поэтому:

- при обновлении или блокировке пользователя вызывается `invalidateUsersCache`;
- при создании, обновлении и удалении товара вызывается `invalidateProductsCache`.

Проверка работы кэширования

Протестируем работу кэша.

Сначала выполним запрос, например, на получение списка товаров:

```
GET /api/products
Authorization: Bearer <accessToken>
```

При первом обращении данные будут получены сервером и сохранены в Redis. Ответ может выглядеть так:

```
{
  "source": "server",
  "data": [
    {
      "id": "1",
      "name": "Ноутбук",
      "price": 75000,
      "description": "Игровой ноутбук"
    }
  ]
}
```

При повторном обращении к этому же маршруту в течение 10 минут данные будут возвращены уже из Redis:

```
{
  "source": "cache",
  "data": [
    {
      "id": "1",
      "name": "Ноутбук",
      "price": 75000,
      "description": "Игровой ноутбук"
    }
  ]
}
```

Аналогично работает кэширование для пользователей и отдельных товаров по id.

Если после этого изменить товар через `PUT /api/products/:id`, соответствующий кэш будет удалён. Следующий `GET`-запрос снова обратится к серверу, а затем обновит кэш.

Пример запуска Redis

Для работы приложения необходимо, чтобы Redis был запущен локально на порту `6379`.

Например, при наличии Docker можно использовать команду:

```
docker run -d --name redis-cache -p 6379:6379 redis
```

После этого можно запускать сервер приложения.

Практическое задание

Необходимо доработать программу из практического занятия №11: добавьте кэширование запросов для маршрутов, перечисленных в таблице:

Маршрут	Метод	Время кэша	Описание маршрута
/api/users	GET	1 минута	Получить список пользователей
/api/users/:id	GET	1 минута	Получить пользователя по id
/api/products	GET	10 минут	Получить список товаров
/api/products/:id	GET	10 минут	Получить товар по id

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.